Multi-Scale Temporal Graph Contrastive Embedding for Urban Region Representation

Yue Li[a], Xinzheng Niu[a,*], Jiahui Zhu[b], Shuai Wen[a], Fan Min[c]

[a]*School of Computer Science and Engineering,University of Electronic Science and Technology of China, chengdu, 611731, China*
[b]*School of Information and Software Engineering,University of Electronic Science and Technology of China, chengdu, 611731, China*
[c]*School of Computer Science, Southwest Petroleum University, chengdu, 610500, China*

Abstract

Regional feature extraction from human mobility data and geographic information is essential for downstream urban management tasks such as crime prediction, check-in prediction, and land use clustering. Existing approaches often focus on the spatial dimensions of urban areas, or analyze temporal dynamics from a single scale perspective. However, they frequently fall short in terms of capturing the intricacies of urban dynamics, due to their inadequate characterization of the temporal heterogeneity within regions. In this paper, we propose a scheme called Multi-Scale Temporal Graph Contrastive Embedding (MTGC), which models human mobility patterns across various time scales to effectively capture local and dynamic variations. First, we conduct independent modeling at three temporal scales: the day scale, the week scale and the month scale. At the day and week scales, we construct spatio-temporal sequence networks to capture local dynamic representations of cities. For the month scale, we utilize a heterogeneous multi-relational network to extract global static representations of urban regions. Next, we propose an autoencoder-based contrastive learning module for multi-scale integration, leveraging the principles of ensemble learning to enhance the overall representation capability. The autoencoder preserves relationships between different regions at the same scale, while contrastive learning fa-

*Corresponding author
Email addresses: liyueeugenia@outlook.com (Yue Li), xinzhengniu@uestc.edu.cn (Xinzheng Niu)

cilitates interaction between representations of the same region across different scales. Experiments on real-world datasets demonstrate that MTGC significantly outperforms existing methods in regard to capturing regional heterogeneity, thus highlighting its potential for more accurate urban region analysis.

Keywords: Urban Region Representation, Graph neural Network, Ensemble learning, Spatio-temporal data mining

1. Introduction

The global urbanization process is rapidly advancing, with 68% of the population being expected to live in urban areas by 2050 [1]. This surge in urbanization brings significant challenges [2], such as overcrowding, resource scarcity, and infrastructure strain [3]. In this context, regional feature extraction from human mobility data and geographic information is essential for effective urban management. By understanding the spatial structure and dynamics of cities, this approach reveals critical relationships between infrastructure, human activity, and urban form. It not only aids in efficient resource management, but also supports downstream tasks such as crime prediction, check-in prediction, and land use clustering, ultimately enhancing urban sustainability and well-being [4].

Current approaches to urban area representation primarily characterize city regions from spatial and temporal perspectives. Methods with a spatial focus, such as MV-PN [5], ROMER [6], and ReCP [7], have made significant strides in capturing spatial features of urban regions. However, these methods often overlook the dynamic and fluid nature of cities which limits their ability to reflect the real-world complexity. Temporally-informed methods such as HDGE [8], MGFN [9], and GraphST [10] incorporate time dynamics in their models, contributing valuable insights into temporal patterns in urban environments. However, these approaches often rely on single-scale temporal analysis, which may not effectively capture the complex and multi-faceted temporal variations inherent in urban settings. Consequently, there is a clear opportunity for existing methods to be enhanced by more effectively capturing the dynamic temporal characteristics of urban areas [11, 12].

In this paper, we propose a novel multi-scale contrastive learning framework for urban region representation. We carry out independent modeling at three temporal scales: the day scale, the week scale, and the month scale.

At the day and week scales, which focus on local dynamics, we first segment time periods based on human activity patterns. Within each segment, we use Graph Convolutional Networks (GCNs) [13] to capture spatial features and encode temporal information using frequency functions. At the month scale, which focuses on global static representations, we first characterize multiple spatial relationships using a top-K approach, and then use heterogeneous graph fusion to share multi-relational representations, thereby obtaining comprehensive month-scale representations.

To effectively integrate information across these temporal scales, we introduce an autoencoder-based contrastive learning module. This design is motivated by multi-perspective comparative learning [14, 15], which excels at aligning representations from different views of the same region. In our case, the same region observed at day, week, and month scales form multiple views that capture distinct but complementary dynamics. Contrastive learning encourages the alignment of these views, promoting consistency across temporal resolutions and enabling robust cross-scale interaction. However, solely focusing on aligning the same region across scales may risk losing the internal structural relationships among different regions within each scale. To address this, we further incorporate an autoencoder, which reconstructs the embeddings and helps preserve scale-specific structural information during the integration process.

Experiments are conducted on publicly available datasets from New York and San Francisco to evaluate model performance across diverse downstream tasks. The model demonstrates strong performance on dynamic tasks such as bike-sharing demand forecasting, highlighting its strength in capturing temporal dynamics. For static tasks, the integrated multi-scale framework achieves robust adaptability across diverse scenarios, while individual scales maintain specialized effectiveness in their respective domains. This dual capability ensures both comprehensive coverage and task-specific precision, advancing the development of versatile urban analytics solutions.

The remainder of this paper is organized as follows: Section 2 provides an extensive review of related work, and positions our study within the existing literature. In Section 3, we formally define the problem and establish the theoretical framework underpinning our research. Section 4 details the proposed methodology, and Section 5 presents the experimental setup, followed by a comprehensive analysis of the results. Finally, Section 6 concludes the paper with a discussion of the key findings and suggests avenues for future research.

2. Related Work

The aim of Urban region embedding is to capture various characteristics of regions, such as Points of Interest(POIs) and human mobility patterns, to create meaningful representations. In recent years, various methods have been developed in this field. These methods can be broadly classified into the following categories: (i) Spatial Embedding Methods; and (ii) Spatio-Temporal Embedding Methods.

2.1. Spatial Embedding Methods

These methods focus on characterizing urban spatial features to understand the structure and function of cities. They primarily analyze the physical layout and spatial relationships of regions. For example, ZE-Mob [16] utilizes human mobility data to analyze regional interactions through co-occurrences. While this approach provides valuable insights, it is limited by reliance on a single data source, and may overlook the multifaceted nature of urban environments.

Multi-source approaches have emerged to enhance the capabilities of single-source data by incorporating diverse data types. For example, MV-PN [5] and CGAL [17] combine POI data with human mobility information to enrich regional representations. Urban2vec [18] integrates POI data with street view imagery to generate neighborhood representations. However, despite these advancements, challenges remain in regard to effectively merging diverse data sources. This difficulty primarily arises from the heterogeneity in spatial, temporal, and semantic properties across modalities, which leads to misalignment and inconsistent feature spaces during the fusion process [19]. In particular, temporal data such as mobility flows often contain rich but noisy patterns, while spatial data like POIs or images are more static but semantically dense, making it non-trivial to align them effectively. In addition to simple data integration, Region2Vec [20] and MVURE [21] use Graph Attention Networks to capture the interdependencies in urban relationships, updating information from individual graphs and fusing them to obtain more comprehensive embeddings. Region2Vec adopts an encoder-decoder architecture to achieve this, whereas MVURE applies self-attention mechanisms and adaptive weighting. ReMVC [22] introduces contrastive learning to address the limitations of previous reconstruction-based methods by contrasting intra-view and inter-view representations to learn more

effective embeddings. HAFusion [23] further refines the process of multi-source data fusion by applying a dual-feature attention module designed for high-order feature correlation learning, which can be integrated into existing models.

The alignment of regional representations with downstream tasks has also been explored. HREP [24] focuses on this alignment through prefix tuning, which enhances task-specific learning outcomes. It is evident that spatial feature extraction advanced the understanding of urban environments. However, this spatially-oriented approach has certain limitations, such as an insufficient capacity to capture the dynamic changes of regions, which may lead to a less comprehensive understanding of urban functions.

2.2. Spatial-Temporal Embedding Methods

These methods incorporated temporal features to capture dynamic changes and periodic patterns. HDGE [8] introduces a region embedding approach that integrates temporal dynamics with multi-hop transfer. However, it mainly relies on predefined geographical proximity relationships, which may be inadequate for complex mobility patterns.

To address this limitation, MGFN [9] employs an automatic clustering method to better capture mobility patterns. It combines a graph structure with spatio-temporal similarity and enhances embedding learning through a multi-level cross-attention mechanism. Despite these advancements, MGFN still faces challenges related to data noise and heterogeneity.

Further improvements are introduced in AutoST [25] and GraphST [10], which represent time-aware regional traffic models. These models decompose traffic data into multiple time periods to reflect dynamic changes. They utilized a variational graph autoencoder along with random walk techniques for automatic spatio-temporal enhancement to address issues of data noise and heterogeneity more effectively.

3. Problem Statement

Urban Region. An urban region R refers to the geographical space within a city that is partitioned into a set of non-overlapping subregions $\{r_1, r_2, \ldots, r_{|N|}\}$, where $|N|$ denotes the total number of subregions.

Human Mobility. Human mobility is defined as assemblage of individual trajectories within the partitioned study area R. Specifically, the trajectory set can be represented as $\mathcal{T} = \{t_i | t_i = (r_s^i, r_d^i, t_s^i, t_d^i)\}$, where each

trajectory t_i encompasses the origin subarea r_s^i, the destination subarea r_d^i, the departure time t_s^i, and the arrival time t_d^i.

Geographic Neighbor Information. This notion represents the spatial relationships between a region and its adjacent regions. Formally, the set of geographic neighbors is denoted as $\mathcal{N} = \{\mathbf{n}_1, \mathbf{n}_2, \ldots, \mathbf{n}_{|N|}\}$, where $\mathbf{n}_i = \{r_1, r_2, \ldots, r_{|\mathbf{n}_i|}\}$ describes the geographic neighbor for the urban region r_i.

Urban Region Embedding. Given sets of human mobility data $\mathcal{T}$ and geographic neighbor information $\mathcal{N}$, our goal is to learn a set of low-dimensional embeddings for urban regions, denoted as $\mathcal{E} = \{\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_{|N|}\}$, where $\mathbf{e}_i \in \mathbb{R}^d$ represents the embedding of the i-th region and d is the embedding dimension. These embeddings are designed to capture the spatial and temporal dependencies among various regions and time slots, and serve as the foundation for various downstream urban applications.

4. Methodology

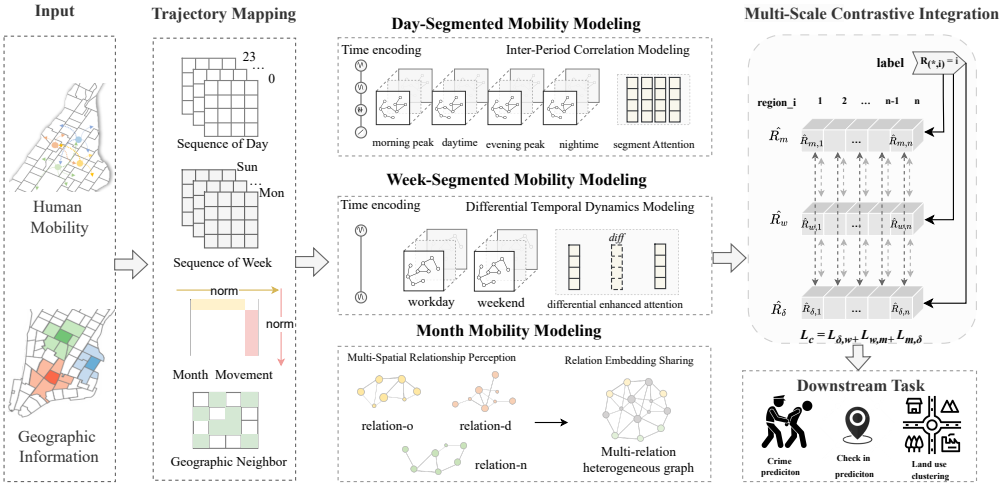


Figure 1: The overall framework of MTGC.

In this section, we propose a novel model called MTGC to capture comprehensive urban region representations by leveraging multi-scale temporal data and multi-view contrastive learning. Figure 1 shows the structure of MTGC with three modules: human mobility mapping, multi-scale modeling, and multi-scale contrastive integration. First, the human mobility mapping

module processes human mobility data and maps them to three temporal scales: the day scale, the week scale, and month scale. These time series provide a detailed and multi-dimensional view of traffic patterns. Next, the multi-scale modeling module independently analyzes each temporal scale. The day scale module captures fine-grained mobility patterns, whereas the week scale module distinguishes urban region functions by differentiating between weekday and weekend traffic patterns, and the month scale module models global interactions between regions, with a focus on long-term spatial relationships. Finally, the multi-view contrastive learning module integrates the representations at the day, week, and month scales, leveraging cross-view interactions to generate comprehensive and robust region embeddings, which are then applied to various downstream tasks.

4.1. Trajectory Mapping

To support multi-scale mobility modeling, we perform trajectory mapping based on the human mobility set $\mathcal{T}$. This process yields three movement representations: daily sequence, weekly sequence, and month movement. Additionally, we encode the geographic neighbor information $\mathcal{N}$ into a binary adjacency matrix to capture spatial proximity.

The sequence of day is defined as $\mathcal{H} = \{\mathbf{H}^{(0)}, \mathbf{H}^{(1)}, \ldots, \mathbf{H}^{(23)}\}$, where each matrix $\mathbf{H}^{(t)} \in \mathbb{R}^{|N|\times|N|}$ represents the movement flow between regions during hour t. Specifically, the entry $\mathbf{H}^{(t)}_{i,j}$ denotes the number of trajectories from region r_i to r_j departing within hour t. The sequence of week $\mathcal{S} = \{\mathbf{S}^{(0)}, \ldots, \mathbf{S}^{(6)}\}$ follows the same construction, with each $\mathbf{S}^{(d)}$ capturing daily movement patterns over the seven days.

To capture long-term mobility patterns, we construct a month movement matrix $F \in \mathbb{R}^{|N|\times|N|}$, where F_{ij} denotes the number of trajectories from region r_i to region r_j aggregated over the entire month. To quantify the relative importance of each region as source and destination, we perform row-wise and column-wise normalization, resulting in two matrices $\mathcal{O} = [\mathcal{O}_{ij}]_{|N|\times|N|}$ and $D = [D_{ij}]_{|N|\times|N|}$:

$$\mathcal{O}_{ij} = \frac{F_{ij}}{\sum_{k=1}^{|N|} F_{ik}}, \quad \mathcal{D}_{ij} = \frac{F_{ij}}{\sum_{k=1}^{|N|} F_{kj}}. \tag{1}$$

The geographic adjacency matrix $\mathcal{A} \in \{0,1\}^{|N|\times|N|}$ is constructed based on $\mathcal{N}$, where $\mathcal{A}_{i,j} = 1$ if region $r_j \in \mathbf{n}_i$, and $\mathcal{A}_{i,j} = 0$ otherwise. This matrix enables spatial structure to be integrated into downstream modeling.

4.2. Day-Segmented Mobility Modeling

Human social activities follow consistent temporal rhythms across different periods of the day, forming the theoretical foundation for temporal segmentation in spatio-temporal modeling. Figure 2 illustrates the aggregated human mobility patterns over a typical day, revealing clear differences in activity intensity and transitions across four distinct temporal intervals: morning peak, daytime, evening peak, and nighttime.

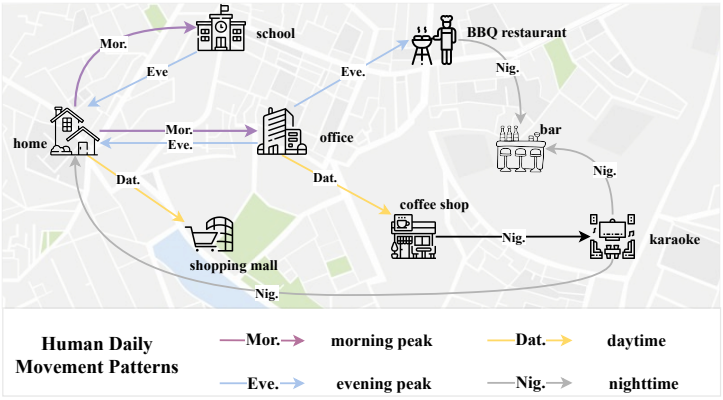


Figure 2: Human mobility follows distinct rhythms: commuting peaks (7–9 a.m., 5–7 p.m.), work hours (9–5 p.m.), and nighttime rest (8–6 a.m.), forming the basis for temporal encoding.

To capture the regional relationships within and between time periods, we partition the day-scale modeling into two components. Figure 3 depicts the overall structural diagram of the day scale modeling. Herein, the intra-period modeling employs the frequency function to differentiate the periods, and the inter-period representation interaction fusion is facilitated through the period aggregation and attention mechanism.

4.2.1. Period-Specific Position Embedding

Human activities over a certain period of time possess significant efficiency in regional discrimination. For instance, some areas still experience high traffic volumes during the night, such as nightlife districts and late-operating commercial zones. These areas often involve increased public activity under low visibility and limited supervision, which has been statistically associated with elevated risks of criminal incidents (e.g., theft, altercations, or disturbances) [26]. In line with human schedules, we partition the 24 hours in a day

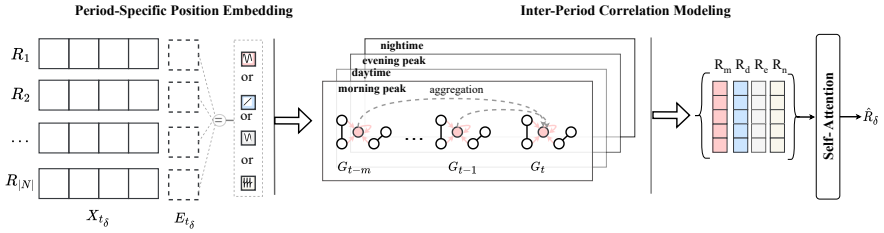


Figure 3: Day-Segmented mobility modeling framework. Each region feature X_{t_δ} is combined with a temporal embedding E_{t_δ} that depends on the period of t_δ. All regions share the same temporal embedding within the same period.

into four periods: the morning peak (7:00–10:00), the daytime (10:00–17:00), the evening peak (17:00–20:00), and the nighttime (20:00–6:00).

To ensure that different periods of the day are distinguishable in temporal encoding, we employ four distinct time-frequency encoding functions, each tailored to the behavioral characteristics of a specific time segment. This design is inspired by the positional embeddings used in Transformer [27], where sine and cosine functions are used to provide explicit positional embedding.

In our setting, the morning and evening peaks exhibit similar structures but occur at different times of day. To distinguish them, we apply the sine function to the morning peak and the cosine function to the evening peak, leveraging the phase difference between these two functions to encode comparable but temporally shifted patterns. For nighttime periods, which are typically characterized by sparse events and low-intensity activity, we use the tangent function. Its ability to emphasize extreme values and model gradual transitions makes it suitable for capturing the subtle variations during these quiet hours. For the relatively steady daytime periods, we adopt a linear encoding to preserve absolute time progression and reflect more stable behavioral patterns.

Formally, the temporal encoding function E_{t_δ} is given by

$$E_{t_\delta} = \begin{cases} \sin(\Delta t_\delta \cdot f_{\text{morning-peak}} \cdot \pi), & t_\delta \in \text{morning peak;} \\ \cos(\Delta t_\delta \cdot f_{\text{evening-peak}} \cdot \pi), & t_\delta \in \text{evening peak;} \\ \tan(\Delta t_\delta \cdot f_{\text{nighttime}} \cdot \pi), & t_\delta \in \text{nighttime;} \\ \frac{\Delta t_\delta \cdot f_{\text{daytime}} \cdot \pi}{24}, & t_\delta \in \text{daytime.} \end{cases} \tag{2}$$

Here, Δt_δ represents the time difference at each time step, $f_{\text{morning-peak}}$, $f_{\text{evening-peak}}$, $f_{\text{nighttime}}$, and f_{daytime} are the learnable frequency parameters for

each period.

4.2.2. Inter-Period Correlation Modeling

After encoding the temporal period for each time step, we apply GCNs to capture the spatial relationship. Each region in the city can be regarded as a node in the traffic spatial network, and the flow between regions can be regarded as a directed weighted edge, when constructing the time slice graph G_t. GCN uses the graph structure of G_t to capture the spatial dependency by transferring and aggregating information from adjacent nodes. The node embedding H_{t_δ} after graph convolution can be denoted as

$$H_{t_\delta} = \mathrm{GCN}(X_{t_\delta} \| E_{t_\delta}, A_{t_\delta}), \quad t_\delta \in \{0, 1, \ldots, 23\}, \tag{3}$$

where X_{t_δ} is the node feature at each time step, E_{t_δ} is the temporal encoding, and A_{t_δ} represents the adjacency matrix of the graph, $\|$ denotes the feature concatenation operation.

The study of interrelation between time periods is equally important, as different time periods are physically continuous and there are causal links between them. These links between study periods help to form a comprehensive representation containing integrated information. After processing the extracted information separately to give time-step representations with GCN, we aggregate the features for each time period as

$$R_\delta = \|_{s=1}^{S} H_s, \quad H_s = \frac{1}{|T_s|} \sum_{t_\delta \in T_s} H_{t_\delta}, \tag{4}$$

where $s \in \{\text{morning\_peak}, \text{evening\_peak}, \text{daytime}, \text{nighttime}\}$, $||$ denotes concatenation operation, S is the number of periods, $|T_s|$ is the number of time steps in period s.

Prior studies have applied Mixture-of-Experts (MoE) to decompose time-series data into components such as trend and seasonality, with each expert specializing in a distinct temporal pattern [28, 29]. These methods typically follow a segmented modeling paradigm, dividing the input sequence into predefined intervals prior to modeling. In contrast, our method adopts a unified and fine-grained modeling strategy that preserves temporal continuity throughout the entire sequence. Specifically, we model the entire 24-hour cycle as a sequence of temporal graphs, each corresponding to one hour and capturing the structural relationships among regions observed during that

time. This design allows the model to retain smooth transitions and contextual dependencies across adjacent hours, which are often disrupted by early segmentation in MoE-based designs. After obtaining these fine-grained representations, we aggregate them into semantically coherent periods, such as morning peak or nighttime. To capture dependencies among these higher-level temporal units, we further apply a self-attention mechanism over the aggregated period representations as

$$Q_\delta = W_{Q_\delta} R_\delta, \quad K_\delta = W_{K_\delta} R_\delta, \quad V_\delta = W_{V_\delta} R_\delta, \tag{5}$$

where Q_δ, K_δ, and V_δ represent the query, key, and value matrices, respectively. W_Q, W_K, and W_V are learnable parameter matrices, and R_δ is the concatenated feature matrix of the segment.

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V, \tag{6}$$

$$\hat{R}_\delta = Mean(Attention(Q_\delta, K_\delta, V_\delta)), \tag{7}$$

where d_k is the dimension of the key matrix, and $Mean(\cdot)$ represents the averaging operation on the sequence output by the self-attention mechanism.

4.3. Week-Segmented Mobility Modeling

The differences in human mobility patterns between weekdays and weekends offer valuable insights into the functional characteristics of urban areas. Analyzing traffic flow variations throughout the week helps in identifying areas dedicated to work or leisure, or those with multi-functional attributes. Some areas have higher traffic on weekdays, indicating a focus on work-related activities, while others see increased traffic on weekends for recreational purposes. In addition, some areas maintain consistently high traffic volumes throughout the week, reflecting diverse functions.

To enable the model to effectively distinguish regional functional diversity, we approach the week scale from three perspectives: weekday, weekend, and their differential aspects. Figure 4 presents a structural diagram of the week scale. Firstly, a time coding mechanism is employed to differentiate between weekday and weekend periods. Following the aggregation process, the model integrates both original features and differential features to develop a comprehensive representation of the week scale.

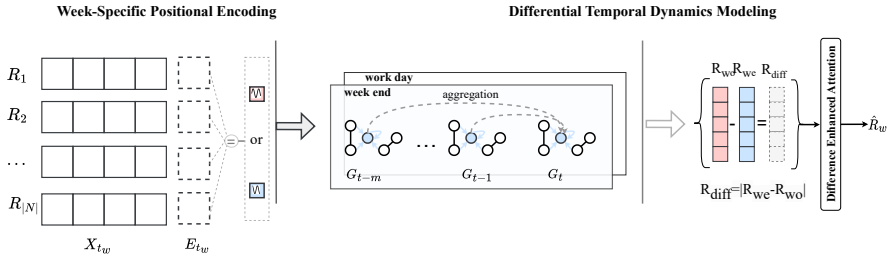


Figure 4: Week-Segmented mobility modeling framework. Each region feature X_{t_w} is combined with a temporal embedding E_{t_w} that depends on the period of t_w. All regions share the same temporal embedding within the same period.

4.3.1. Week-Specific Positional Encoding

Similarly to the modeling used in the previous period, the weekly sequence of slices does not directly incorporate period discrimination information. We therefore utilize sin and cos to encode the intra-week and weekend periods

$$E_{t_w} = \begin{cases} \sin(\Delta t_w \cdot f_{\text{workday}} \cdot \pi), & t_w \in \text{workday}; \\ \cos(\Delta t_w \cdot f_{\text{weekend}} \cdot \pi), & t_w \in \text{weekend}, \end{cases} \tag{8}$$

where t_w denotes the time step of week and Δt_w denotes the time difference at each time step. $f_{workday}, f_{weekend}$ is the learnable frequency parameters. Then GCN is adopted in the same way to model the spatial dependence as

$$H_{t_w} = \text{GCN}(X_{t_w} \| E_{t_w}, A_{t_w}), \quad t_w \in \{0, 1, \ldots, 6\}. \tag{9}$$

However, unlike the process described above, modeling at the week scale not only needs to reflect the association between time periods, but also needs to consider the urban function from the perspective of differences between time periods. We initially aggregate H_{t_w} based on whether t_w falls on weekdays or weekends, thereby obtaining the respective representations R_{wo} for workdays and R_{we} for weekends.

$$R_{wo} = \frac{\sum_{t_w \in wo} H_{t_w}}{N_{wo}}, \quad R_{we} = \frac{\sum_{t_w \in we} H_{t_\omega}}{N_{we}}, \tag{10}$$

where $wo = \{0, 1, 2, 3, 4\}$ is the set of weekday time steps and $we = \{5, 6\}$ is the set of weekend time steps. Then, the difference representation R_{diff} is given by

$$R_{\text{diff}} = |R_{wo} - R_{we}|. \tag{11}$$

4.3.2. Differential Temporal Dynamics Modeling

To allow the model to synthesize the raw and differential features, we splice the differential features with the raw features for the self-attention mechanism. The self-attention mechanism integrates the dynamic relationship between time periods through global dependency modeling capability. The week view representation $\hat{R_w}$ is therefore obtained as

$$\hat{R_w} = Mean(\text{SelfAttention}([R_{\text{diff}}, R_{\text{wo}}, R_{\text{we}}])). \tag{12}$$

4.4. Month Mobility Modeling

The month-scale modeling aims to capture globally persistent spatial patterns of urban mobility, complementing the day and week modules that focus on temporal dynamics. Unlike shorter time scales that exhibit significant variability, spatial structures such as road networks and functional zone characteristics remain stable over a month-long period [30]. This spatial invariance enables effective modeling of city-wide mobility patterns using single-month data. By focusing on these stable spatial features, the month-scale module provides a robust representation of fundamental urban mobility structures while maintaining computational efficiency. Figure 5 illustrates the modeling structure for the month scale. This model begins by approaching spatial relationships from a single perceptual perspective. Subsequently, a multi-relational heterogeneous graph fusion module is employed to capture common embedding across multiple spatial relations, thereby yielding a comprehensive representation without requiring data from multiple months.

4.4.1. Single-view Relationship Perception

Regions that receive traffic from the same sources or which direct traffic to the same destinations typically serve similar functions, and are considered spatially close from the perspective of human mobility. Given the source correlation $\mathcal{O}$, destination correlation $\mathcal{D}$, and geographic neighborhood correlation $\mathcal{A}$, we construct a multi-relation heterogeneous graph $G = (\mathcal{V}, \mathcal{E}, \mathcal{R})$, where $\mathcal{R}$ denotes the set of edge types, including source edges, destination edges, and geographical neighbor edges. Each edge type connects nodes with non-zero values in the adjacency matrix.

Next, we employ GCN in the same way as previously to aggregate the information on the neighbor nodes and edges. However, in contrast to the dense flow graphs processed on a daily and weekly basis, the month-scale

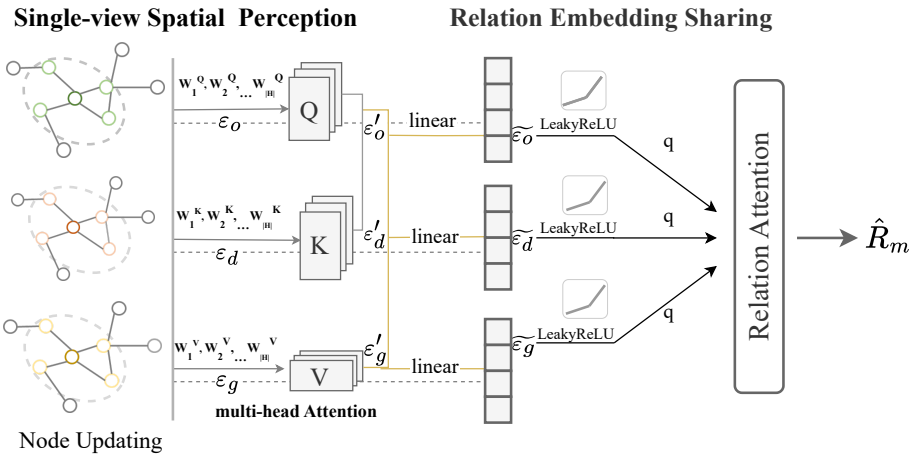


Figure 5: Month Mobility Modeling.

spatial relationship graph is a sparse graph that retains only the top-k associations. Sparse graphs are characterized by limited semantic content, which can readily result in underfitting issues. To address this, we stack multiple GCN layers, enabling the model to learn deeper, more complex patterns and relationships within the graph structure.

Given the initial node embeddings $\varepsilon_\nu^{(0)} = \{\mathbf{e}_1^{(0)}, \mathbf{e}_2^{(0)}, \ldots, \mathbf{e}_n^{(0)}\}$ and the initial relation embeddings $\varepsilon_r^{(0)} = \{\mathbf{e}_s^{(0)}, \mathbf{e}_d^{(0)}, \mathbf{e}_g^{(0)}\}$, the multi-layer GCN updates the node information of l-th layer through

$$\mathbf{e}_i^{(l)} = \sigma\left(\sum_{j \in \mathcal{N}_r(i) \cup \{i\}} \frac{\mathbf{W}^{(l)}(\mathbf{e}_j^{(l-1)} \circ \mathbf{e}_r^{(l-1)})}{\sqrt{|\Gamma_r(i)|} \cdot \sqrt{|\Gamma_r(j)|}}\right), \tag{13}$$

where $\mathbf{W}^{(l)}$ is a learnable parameter of l-th layer, σ denotes the LeakyReLU activation function. $\Gamma_r(\cdot)$ represents the top-k neighborhood under relation r, and $\circ$ denotes the Hadamard product. After updating node information under the respective relations $\mathcal{O}$, $\mathcal{D}$, and $\mathcal{N}$, we obtain $\varepsilon = \{\varepsilon_o, \varepsilon_d, \varepsilon_g\}$.

4.4.2. Relation Embedding Sharing

After obtaining the representations for each spatial relationship, we perform cross-relational interaction. Following the process in HREP [24], we first correlate the different attributes of the same region and use a multi-head

attention mechanism to enable interaction across different spatial views.

$$\varepsilon' = (\Big\|_{h=1}^{H} Attention(\varepsilon \mathbf{W}_h^Q, \varepsilon \mathbf{W}_h^K, \varepsilon \mathbf{W}_h^V))\mathbf{W}^I, \tag{14}$$

where $\|$ represents the concatenation operation, H is the number of attention heads. $\mathbf{W_h}^Q$,$\mathbf{W_h}^K$,$\mathbf{W_h}^V$ are trainable parameter for h-th head, used for query, key, and value transformations respectively. $\mathbf{W}^I$ is a learnable parameter for transformation. We then integrate the distinct features of each view into the global information through learnable linear interpolation by

$$\widetilde{\varepsilon_r} = c_r \varepsilon_r' + (1 - c_r)\varepsilon_r, \tag{15}$$

where c_r is the linear interpolation weight, $\varepsilon_r \in \varepsilon$ is the node representation set of relation r after GCN, and $\varepsilon_r' \in \varepsilon'$ is the node representation set of relation r after multi-head attention.

we employ a relation-specific attention mechanism to integrate the representations derived from each relation. Given the interpolated embedding $\widetilde{\varepsilon_r} = \left\{e_j^i\right\}_{j=1}^{|N|}$, the month-view representation $\hat{R_m}$ is computed as

$$w_r = \frac{1}{|N|} \sum_{i=1}^{|N|} \mathbf{q}^T \cdot \sigma(\mathbf{W}\mathbf{e}_i^r + \mathbf{b}), \tag{16}$$

$$\tilde{w_r} = \text{softmax}(w_r), \quad \hat{R}_m = \sum_{r=1}^{3} \tilde{w_r} \cdot \tilde{\varepsilon_r}, \tag{17}$$

where σ is the LeakyReLU activation function, and $\mathbf{q}, \mathbf{W}, \mathbf{b}$ are learnable attention vectors for all relations.

4.5. Multi-scale Contrastive Integration Module

After extracting the regional features at the scale of days, weeks, and months, the key issue is how to effectively fuse these features from different perspectives [3]. We designs a novel multi-scale learning model based on an attention mechanism. As depicted in Figure 6, the model is composed of encoders, decoders, and logarithmic attention modules, which aim to retain the semantic and structural information of features at each scale while fusing multi-scale features.

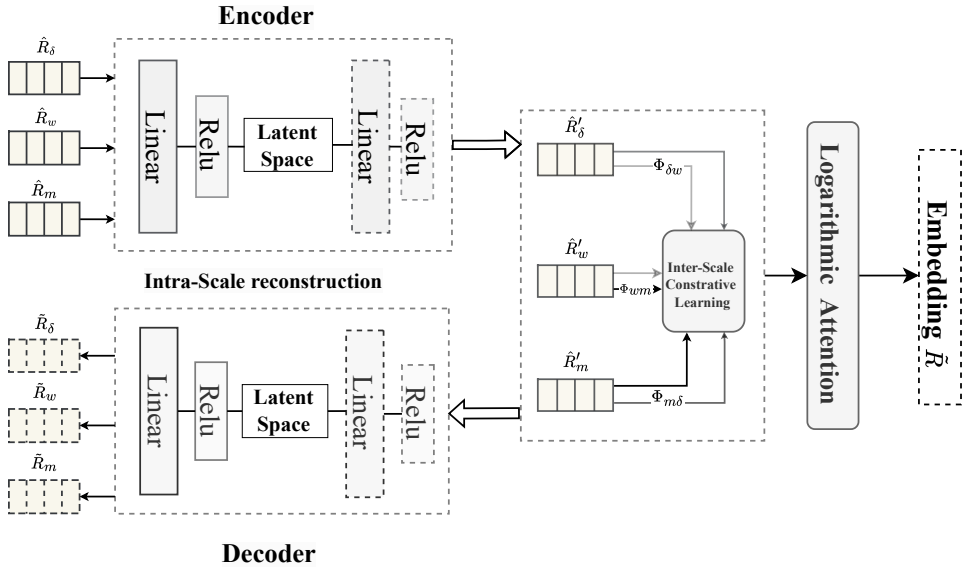


Figure 6: Multi-Scale Contrastive learning framework.

4.5.1. Encoders and Decoders

For each scale-specific feature, encoder projects high-dimensional input features into a low-dimensional latent space by

$$\hat{R}'_i = \text{Relu}(W_{i2} \cdot \text{Relu}(W_{i1} \cdot \hat{R}_i + b_{i1}) + b_{i2}), \tag{18}$$

where $\hat{R}_i \in \mathbb{R}^{d_1}$ denotes input features at the i-th scale, and $\hat{R}'_i \in \mathbb{R}^{d_2}$ represents the corresponding latent feature in a lower-dimensional space. Here, d_1 and d_2 are adjustable hyperparameters determining the dimensionality of the input and latent representations, respectively. The weight matrices W_{i1} and W_{i2} are associated with biases b_{i1} and b_{i2}, respectively.

The decoder architecture mirrors that of the encoder, yet its objective is to maintain the connectivity of regional representations at the same scale. Consequently, it accepts the low-dimensional representation produced by the encoder and projects it back by

$$\tilde{R}_i = \text{Relu}(W_{i4} \cdot \text{Relu}(W_{i3} \cdot \tilde{R}'_i + b_{i3}) + b_{i4}), \tag{19}$$

where $\tilde{R}_i \in \mathbb{R}^{d_2}$ is the low-dimensional representation obtained from the encoder, and $\tilde{R}'_i \in \mathbb{R}^{d_1}$ represents the reconstructed feature in the original space. The parameters d_1 and d_2 maintain consistency with the encoder

dimensions. The weight matrices W_{i3} and W_{i4} are learnable, along with biases b_{i3} and b_{i4}.

4.5.2. Logarithmic Attention Mechanism

While retaining the information on the features at each scale, it is essential to perform cross-scale decision integration to obtain a comprehensive representation. We therefore introduce a logarithmic attention mechanism (LAM) to dynamically adjust the fusion weights of features during the fusion process, to capture the correlations between features more effectively. Linearly transform the input multi-scale sequence R_l, the query matrix Q_l, key matrix K_l and value matrix V_l is given by

$$Q_\iota = W_{Q_\iota} R_\iota, K_\iota = W_{K_\iota} R_\iota, V_\iota = W_{V_\iota} R_\iota. \tag{20}$$

Here, $R_\iota = [\hat{R}'_\delta, \hat{R}'_w, \hat{R}'_m]$ is the stacked multi-scale input feature matrix. To suppressing low-probability noise and enhancing the main features, we use logSoftmax to adjust the attention weights A by

$$A = \text{log-Softmax}(\frac{Q_l K_l^T}{\sqrt{d_k}}), \quad \text{log-Softmax}(Z) = Z - \log\left(\sum_{j=1}^{n} \exp(\mathbf{Z}_j)\right), \tag{21}$$

among which log probabilities amplifies larger attention weights and diminishes smaller ones. The final representation $\tilde{R}$ of MTGC is given by

$$\tilde{R}_l = A \cdot V_l, \quad \tilde{R} = \frac{1}{3}\sum_{i=1}^{3} \tilde{R}_i, \tag{22}$$

where $\tilde{R}_i$ represents the i-th element in the sequence $\tilde{R}_l$.Through the use of LAM, features at different scales are fused in a way that preserves their individual characteristics while enhancing feature interaction and integration.

4.6. Learning Objectives

To effectively train our model, we employ a multi-scale integrated learning approach. The objective function is structured into three main components: a temporal sequence loss, a heterogeneous graph multi-task loss, and a multi-view contrastive learning loss. We adopt a staged training strategy. The first two loss functions are applied during the initial training stage to obtain region representations at three independent scales. After completing the first stage and acquiring the individual representations, the multi-view contrastive loss is utilized in the second stage to integrate the multi-scale representations.

4.6.1. Temporal Sequence Loss

The temporal sequence loss encompasses both day and week scales. At each of these scales, we aim to capture the mobility patterns through a KL divergence-based loss function. This loss is time-step dependent, meaning that at each time step, the embeddings for the current and next time steps are used to reconstruct the transition probabilities between regions.

For each hour or day, we compute the embeddings e_t and e_{t+1} for the current and next time steps respectively. These embeddings are used to predict the transition probabilities for both outflow and inflow mobility patterns. The loss is calculated as the sum of the KL divergences between the predicted transition probabilities and the actual mobility matrices. The predicted transition probabilities $p_s(r_j \mid r_i, t)$ and $p_t(r_i \mid r_j, t)$ are computed as

$$p_s(r_j \mid r_i, t) = \frac{\exp(e_t^i \cdot e_{t+1}^j)}{\sum_k \exp(e_t^i \cdot e_{t+1}^k)}, \tag{23}$$

$$p_t(r_i \mid r_j, t) = \frac{\exp(e_{t+1}^j \cdot e_t^i)}{\sum_k \exp(e_{t+1}^j \cdot e_t^k)}. \tag{24}$$

The actual transition probabilities $d_s(r_j \mid r_i, t)$ and $d_t(r_i \mid r_j, t)$ are derived from the flow matrices as

$$d_s(r_j \mid r_i, t) = \frac{F_{ij}(t)}{\sum_k F_{ik}(t)}, \tag{25}$$

$$d_t(r_i \mid r_j, t) = \frac{F_{ji}(t)}{\sum_k F_{jk}(t)}. \tag{26}$$

The overall time series loss L_T is then calculated as the sum of the KL divergence between the predicted and actual transition probabilities across all regions and all time steps

$$\begin{aligned} L_{\text{TS}} = \sum_t \Bigg(\sum_{i,j} \Bigg(& p_s(r_j|r_i,t) \log \frac{p_s(r_j|r_i,t)}{d_s(r_j|r_i,t)} \\ & + p_t(r_i|r_j,t) \log \frac{p_t(r_i|r_j,t)}{d_t(r_i|r_j,t)} \Bigg) \Bigg). \end{aligned} \tag{27}$$

4.6.2. Heterogeneous Graph Multi-Task Loss

At the month scale, we adopt a multi-task learning framework that integrates the movement prediction loss and the triplet loss to preserve the spatial relationships. Unlike the reconstruction using adjacent time step embeddings, the mobility prediction loss performs embedding reconstruction from the source and target functionality separately.

i) Mobility predicted loss: To reconstruct the mobility patterns, we first compute the predicted transition probabilities $\hat{p}_{i,j}$ and $\hat{q}_{i,j}$ by normalizing the inner product of the source and destination embeddings across all possible destinations.

$$\hat{p}_{ij} = \frac{\exp(e_s^i \cdot e_d^j)}{\sum_k \exp(e_s^i \cdot e_d^k)}, \quad \hat{q}_{ij} = \frac{\exp(e_d^j \cdot e_s^i)}{\sum_k \exp(e_d^j \cdot e_s^k)}, \tag{28}$$

where e_s^i is the embedding of region r_i under the source functional relation, e_d^j is the embedding of region r_j under the destination functional relation. Here, the observed movement data between regions r_i and r_j over the month M_{ij} is given by

$$\mathrm{M}_{ij} = \frac{F_{ij}}{\mu_F}, \tag{29}$$

where F_{ij} denotes the traffic flow from region i to region j, μ_F is the mean value of traffic flow matrix F.

The mobility prediction loss compares the predicted and actual mobility distributions using a cross-entropy-based approach as

$$L_{\mathrm{mob}} = \sum_{i,j} \left(-\mathrm{M}_{ij} \log(\hat{p}_{ij}) - \mathrm{M}_{ij} \log(\hat{q}_{ij})\right). \tag{30}$$

ii) Geographical Loss: To capture the spatial relationships, we use a triplet loss which ensures that the embeddings of geographically adjacent regions are closer compared to non-adjacent regions.

$$L_{\mathrm{geo}} = \sum_i \max\{0, \|e_i - e_i^{\mathrm{pos}}\|_2 - \|e_i - e_i^{\mathrm{neg}}\|_2\}, \tag{31}$$

where e_i is the embedding of region i, e_i^{pos} ($resp. e_i^{\mathrm{neg}}$) is the embedding of the positive ($resp. negative$) sample of region i for geographic neighbors. The overall loss for the month scale is a combination of these two losses:

$$L_{\mathrm{HG}} = L_{\mathrm{mob}} + L_{\mathrm{geo}}. \tag{32}$$

4.6.3. Multi-Scale Contrastive Learning Loss

The last component of our loss function is multi-view contrastive learning, which integrates the embeddings learned at day, week, and month scales. This component includes the reconstruction loss and contrastive loss to ensure consistency across different views.

i) Reconstruction Loss: By minimizing the encoder and decoder output feature matrices using the Mean Square Error(MSE), the reconstruction loss preserves the internal correlations of feature vectors at each scale.

$$L_{\text{rec}} = \sum_{i \in \{\delta, w, m\}} \|\tilde{R}_i - \hat{R}'_i\|^2, \tag{33}$$

Here, the terms δ, w, and m represent the day, week, and month scales, respectively. The variable $\hat{R}'_i$ denotes the original features at scale i, which serve as the input to the encoder, and $\tilde{R}_i$ is the reconstructed feature via the decoder.

ii) Contrastive Loss: In order to allow different scale representations belonging to the same region to learn from each other, we consider different scale representations from the same region as positive samples, and the rest as negative samples. A contrastive loss is then used to maximize the distance between the positive samples and to minimize the distance between negative samples. First, the similarity matrix Φ_{ij} between different the i-th scale and the j-th scale is calculated by

$$\Phi_{ij} = \frac{\hat{R}'_i \cdot (\hat{R}'_j)^T}{\tau}, \tag{34}$$

where $\hat{R}'_i$ is the embedding matrix of i-th scale, where $(\cdot)^T$ denotes the transpose of matrix. And τ is the temperature parameter that adjusts the sensitivity of the similarity.

Next, each region is treated as an independent sample, and labels y are generated such that $y_i = i$. Finally, the contrastive loss is computed using the cross-entropy loss function:

$$L_{\text{con}} = \sum_{(i,j) \in \{(\delta,w),(w,m),(m,\delta)\}} \text{CrossEntropy}(\Phi_{ij}, y_i). \tag{35}$$

The overall loss for multi-view contrastive learning combines these two parts:

$$L_{\text{MS}} = \alpha \cdot L_{\text{rec}} + (1 - \alpha) \cdot L_{\text{con}}, \tag{36}$$

where $\alpha \in [0, 1]$ is a learnable weight coefficient that balances the reconstruction loss and contrastive loss.

4.7. Time Complexity Analysis of MTGC

The proposed model MTGC consists of three main components as shown in Figure 1. To provide a detailed analysis of its computational characteristics, we first examine the time complexity of each component individually. For clarity, the symbols specific to this section are defined in Table 1.

Table 1: Symbols used in time complexity analysis.

Symbol	**Description**	**Value**
k_s	The number of scales.	3
k_r	The count of nearest neighbors in each relational similarity matrix.	10
n_{tra}	The number of trajectories considered in the study area.	10^7
T_δ, T_w	Time steps corresponding to day, week scales, respectively.	24, 7
F_s, F_m	The embedding dimension before and after multi-scale integration.	144, 96
n_δ, n_w, n_m	The number of aggregated periods of day, week, month scale.	4, 3, 3
$\|E\|$	The number of edges in the adjacency matrix.	-
L	The number of GCN layers used in month scale.	3
N_{in}, N_{out}	The number of neurons in the encoder and decoder, respectively.	-, -

4.7.1. Trajectory Mapping Time Analysis

In the first phase of MTGC, trajectories are mapped to three independent scales based on their start and end regions as well as timestamps. At each scale, each trajectory undergoes a classification and mapping process. Therefore, the time complexity of trajectory mapping is $O(k_s \cdot n_{\text{tra}})$,where k_s represents the number of scales and n_{tra} denotes the number of trajectories.

4.7.2. Multi Scale modeling Computational Complexity

After obtaining the trajectory mapping results, we independently model the trajectories at three scales: the day scale, the week scale, and the month scale. Correspondingly, the time complexity of this phase is the sum of the time complexities for the three scales, which is given by $O(T_\delta(|N|^2 \cdot F_s))$. Here, T_δ is the number of daily time steps, $|N|$ is the number of regions, and F_s is the embedding dimension of regions.

Proof. i) *At the day scale, we use GCN to embed the traffic graph between regions for each time step. For dense graphs, the complexity of GCN is* $O(|N|^2 \cdot F_s + |N| \cdot F_s^2)$. *Considering* T_δ *time steps, the GCN contributes a total complexity of* $O(T_\delta \cdot (|N|^2 \cdot F_s + |N| \cdot F_s^2))$. *After obtaining the representations for each time step, we apply the self-attention mechanism for inter-period interaction. The self-attention operates with a complexity of* $O(n_\delta^2 \cdot F_s)$, *where* n_δ *is the number of aggregated periods. Since* $|N| > F_s > n_\delta$, *the time complexity at the day scale is* $O(T_\delta \cdot (|N|^2 \cdot F_s + |N| \cdot F_s^2) + n_\delta^2 \cdot F_s)$, *approximated to* $O(T_\delta(|N|^2 \cdot F_s))$.

ii) *The modeling process at the week scale is similar to the day scale, differing only in the number of time steps and aggregated periods. Therefore, the time complexity for this scale is* $O(T_w(|N|^2 \cdot F_s))$, *where* T_w *the number of week time steps.*

iii) *At the month scale, we initially employ GCN to conduct multi-spatial relation embedding. For sparse graphs, the GCN has a complexity of* $O(L(|E| \cdot F_s + |N| \cdot F_s^2))$, *where* L *is the number of GCN layers and* $|E| = |N| \cdot k_r$ *denotes the number of edges in the graph. Since* $k_r < F_s$, *the dominant term in single-view relation perception is* $O(L(|N| \cdot F_s^2))$. *Subsequently, linear interpolation based on multi-head attention and relation attention based on self-attention are applied to the relation embedding sharing. These steps have time complexities of* $O(n_m^2 \cdot F_s)$ *and* $O(n_m \cdot F_s^2)$, *respectively. Since* $|N| > F_s > n_m$, *the time complexity at the month scale is* $O(L(|N| \cdot F_s^2))$.

Considering the above three scales, the total time complexity of the multi-scale modeling phase is $O(T_\delta(|N|^2 \cdot F_s) + T_w(|N|^2 \cdot F_s) + L(|N| \cdot F_s^2))$. *Given that* $T_\delta > T_w$ *and* $|N| > F_s$, *the dominant term across all scales is* $O(T_\delta(|N|^2 \cdot F_s))$. *Therefore, the overall time complexity simplifies to* $O(T_\delta(|N|^2 \cdot F_s))$.

4.7.3. Multi-Scale Contrastive Module Complexity

In the multi-scale integration and fusion phase, we first use autoencoders to reconstruct the semantic relationships within each scale. Afterward, a Logarithmic Attention Mechanism is applied to integrate the multi-scale

representations and obtain the final embedding. The time complexity of multi-scale integration is $O(N_{in} \cdot N_{out})$, where N_{in} is the number of encoder neurons and N_{out} is the number of decoder neurons.

Proof. *The time complexity of this autoencoders is* $O(F_s \cdot N_{in} + N_{in} \cdot N_{out})$. *The core computation of LAM is similar to self-attention, with a time complexity of* $O(k_s^2 \cdot F_m)$, *where* F_m *is the embedding dimension. Therefore, the time complexity of the third phase is* $O(F_s \cdot N_{in} + N_{in} \cdot N_{out} + k_s^2 \cdot F_m)$, *approximated to* $O(N_{in} \cdot N_{out})$.

Considering the above three phases, the overall time complexity of MTGC is $O(k \cdot n_{\text{tra}} + T_\delta(|N|^2 \cdot F_s) + N_{in} \cdot N_{out})$. While the introduction of multi-scale modeling increases the time complexity due to processing across different temporal scales, this trade-off is justified. The approach significantly improves the granularity and comprehensiveness of the learned representations, leading to better region modeling performance. In our experiments, training for 2000 epochs on an NVIDIA 4090 takes approximately 20 minutes, demonstrating that despite the increased complexity, the model remains computationally feasible.

5. Experiments

5.1. Experimental Settings

5.1.1. Data Description

Table 2 summarizes the various real-world datasets collected from the NYC open data portal[1] and the San Francisco open data portal[2].

(1) Census blocks and taxi trip data are used for urban feature extraction. Taxi trip data are used to capture human mobility, and census blocks are used to represent geographical information.

(2) Crime and check-in datasets serve as ground truth for downstream prediction tasks. Each record is associated with a location and timestamp. We aggregate the number of crime reports and check-ins within each region.

(3) Land use dataset is introduced as a reference for clustering evaluation. While actual land use data offer fine-grained functional information, they are defined at a spatial granularity much smaller than our region units. A single region often overlaps multiple land use categories, making it difficult

[1] https://opendata.cityofnewyork.us
[2] https://data.sfgov.org

Table 2: Data Description

Dataset	NYC	SF
Census blocks	Boundaries of 180 regions within Manhattan, New York.	Boundaries of 194 regions within San Francisco.
Taxi trips	Total 9,779,723 records of taxi trips during a month.	Total 539,447 records of taxi trips during a month.
Check-in data	Total 106,902 check-in records spanning more than 200 finely detailed categories.	Total 170,001 check-in records spanning more than 380 finely detailed categories.
Crime data	Total 35,335 crime incidents during a year.	Total 135,217 crime incidents recorded during a year.
Land use	Land use data categorized according to planning divisions, dividing Manhattan into 12 districts.	Planning districts data divides San Francisco into 17 districts.

to assign a consistent label. Therefore, following prior work [9, 21, 24], we adopt municipal planning districts as the ground truth. These districts are delineated by urban planning authorities based on comprehensive land use considerations, and their spatial resolution aligns better with our region-level representations.

5.1.2. Parameter Settings

In this study, the model parameters are set as follows: the number of node initialization features is 250, the number of multi-scale output channels is 144, and the number of final embedded features is 96. The number of time features for the day scale is eight, with an hour time difference of one, and one GCN layer per time step. The number of weekly temporal features is eight, with a day time difference of 24 and one GCN layer. The month scale has three GCN layers.The Adam optimizer is used, with a learning rate of 0.001 and a weight decay of 5e-4. For the fusion model, the learning rate is 0.01 and the weight decay is 5e-5. The total number of training epochs is set

to 2000. The details of implementation can be found in the released code[3].

5.1.3. Baselines

We compare our model with the following 6 baselines:

- **GAE** [31] This employs GCNs to encode node features and reconstruct graph structures through a decoder, achieving unsupervised graph representation learning and obtaining low-dimensional node embeddings.
- **LINE** [32] This approach effectively captures local and global structural features in large-scale networks by preserving both first-order and second-order proximities to generating node embeddings.
- **node2vec** [33] This scheme utilizes a flexible random walk strategy to generate node sequences and learns high-dimensional vector representations of nodes by optimizing a specific objective function, thus reflecting complex relationships between graph structures and nodes.
- **HDGE** [8] In this approach, temporal dynamics and multi-hop transfer relationships between regions are modeled by integrating flow graphs and spatial graphs, thus enabling the learning of region representations.
- **MVURE** [21] This provides a multi-view unified graph representation learning method by combining human mobility data and inherent regional attributes and using a graph attention mechanism to construct comprehensive urban region models.
- **MGFN** [9] In this scheme, different mobility pattern graphs are integrated and a cross-modal attention mechanism is applied to generate comprehensive urban region embeddings.
- **HREP** [24] This is a method for heterogeneous region embedding through prompt learning, which uses a relation-aware GCN and a self-attention mechanism for heterogeneous region embeddings and employs a prefix-tuning approach for prompt learning.
- **ReCP** [7] A multi-view region embedding model that enhances representation consistency through contrastive learning and dual prediction

[3] https://github.com/eugenia-cell/MTGC

across views, maximizing shared information while reducing inconsistency.

5.2. Prediction Task Performance

In our study, we performs two tasks that are widely used for region representation: prediction and clustering. For prediction, we focus on crime and check-in events, and the goal is to accurately predict the number of events based on the learned embeddings. The multi-scale temporal data are only used in the upstream stage to construct region embeddings, aiming to capture spatiotemporal patterns from different temporal resolutions. These embeddings are then used as static feature inputs for downstream tasks. Since the prediction task does not directly involve temporal sequences, but instead operates on fixed embeddings, it can be treated as a standard regression problem. Therefore, we apply k-fold cross validation to ensure robust and reliable evaluation, where the dataset is divided into multiple folds and a Lasso regression model [34] is trained to predict the target variable. We then calculate the Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and coefficient of determination (R^2) to evaluate the accuracy of the model.

Table 3: Crime Prediction Prediction Results of New York and San Francisco.

Crime Prediction	New York City			San Francisco		
	MAE↓	RMSE↓	R^2↑	MAE↓	RMSE↓	R^2↑
GAE [31]	96.55	133.10	0.19	457.54	740.89	0.31
LINE [32]	117.53	152.43	0.06	501.84	808.49	0.18
Node2vec [33]	75.09	104.97	0.49	437.44	677.96	0.42
HDGE [8]	72.65	96.36	0.58	-	-	-
MVURE [21]	65.16	88.19	0.64	372.85	618.77	0.52
MGFN [9]	70.21	89.60	0.63	391.49	592.86	0.56
HREP [24]	65.66	84.59	0.68	350.79	555.95	0.61
ReCP [7]	63.97	85.46	0.67	313.68	569.35	0.59
Ours	**58.29**	**80.15**	**0.71**	**343.75**	**537.25**	**0.64**
Improve	**11.22%**	**5.24%**	**4.41%**	**2.01%**	**3.36%**	**4.92%**

Table 3 and Table 4 summarizes the performance of each model on diverse tasks, and a detailed analysis is as follows. Traditional graph embedding approaches such as LINE and Node2Vec fail to incorporate the semantic context of urban regions, leading to suboptimal performance. Advanced models that

Table 4: Check-in Prediction Results of New York and San Francisco.

Check-in Prediction	New York City			San Francisco		
	MAE↓	RMSE↓	R^2↑	MAE↓	RMSE↓	R^2↑
GAE [31]	498.23	803.34	0.09	690.61	1456.96	0.48
LINE [32]	564.59	853.82	0.08	925.95	1863.37	0.16
Node2vec [33]	372.83	609.47	0.44	825.33	1440.15	0.49
HDGE [8]	399.28	536.27	0.57	-	-	-
MVURE [21]	297.72	495.27	0.63	586.40	1180.98	0.66
MGFN [9]	292.30	451.76	0.69	683.74	1209.34	0.64
HREP [24]	270.28	406.53	0.75	590.19	1185.75	0.66
ReCP [7]	258.02	406.67	0.75	459.66	1092.49	0.71
Ours	**228.01**	**352.04**	**0.81**	**571.57**	**1008.90**	**0.75**
Improve	**15.64%**	**13.40%**	**8.00%**	**-**	**14.91%**	**5.33%**

consider the spatio-temporal dynamics of urban changes, such as HDGE, achieve better success. MVURE and HREP apply an effective heterogeneous data fusion method from the perspective of multi-source data heterogeneity, which further enhances the prediction effect for crime and check-ins. ReCP achieves performance comparable to HREP. This may be attributed to its reliance on contrastive objectives to align multi-view information, which benefits more from data with clearer inter-view consistency and less structural noise. Our model adopts the perspective of multi-scale analysis, and captures both fine-grained and coarse-grained spatio-temporal features, thereby outperforming all state-of-the-art methods.

5.3. Clustering Task Performance

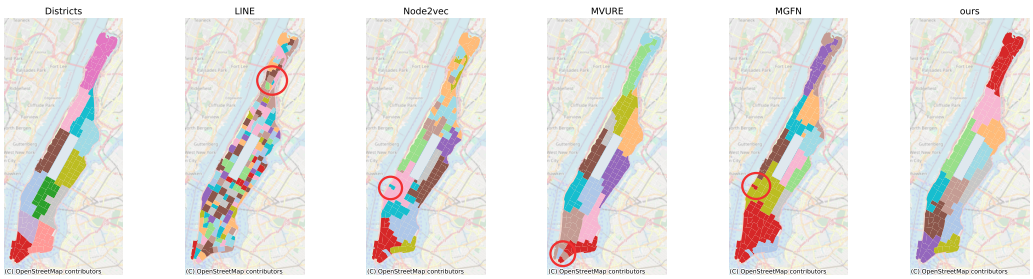


Figure 7: Comparison of land use clustering results and ground truth in Manhattan.

To comprehend the data distribution and evaluate our model's capacity to discriminate features, we carry out clustering on the representation

vectors. By grouping regions based on their learned representations, clustering discloses functional similarities and disparities, which is in line with our objective of classifying regions in accordance with their underlying land-use characteristics. The district division of the community board is used as the ground truth, and Manhattan is divided into 12 districts mainly based on land use. We employ k-means [35] as the clustering algorithm, and the NMI and ARI are utilized to assess the clustering performance.

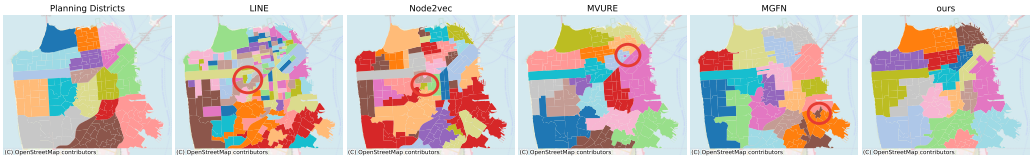


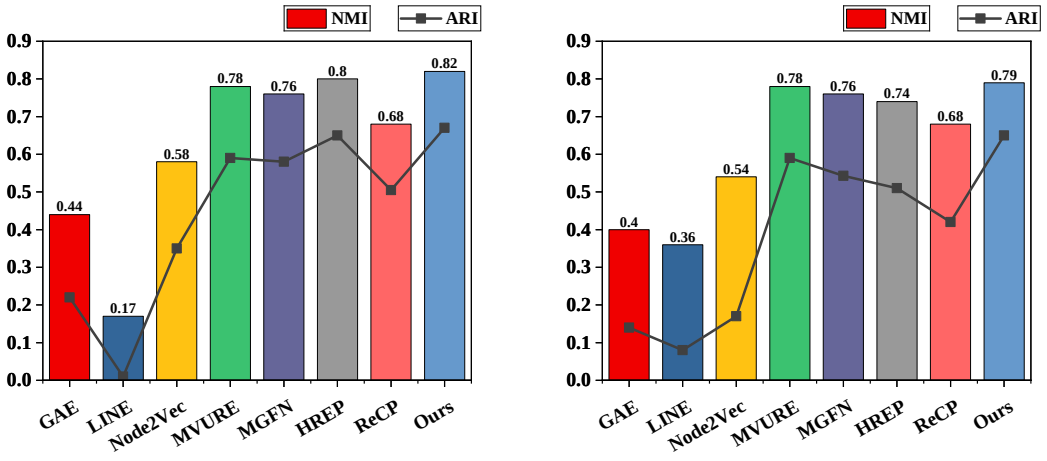
Figure 8: Comparison of land use clustering results and ground truth in San Francisco.

Figure 7 and Figure 8 show a comparison between the different clustering methods and the actual region division. The classical graph embedding algorithm LINE shows mixed categories between adjacent areas, failing to form coherent block structures. Node2vec improves upon LINE by demonstrating clearer block formations, but there remains a substantial gap between the segmented blocks and the actual regions, with many unclear boundaries. The state-of-the-art methods MVURE and MGFN achieve better block segmentation, and more accurately reflect functional urban areas. However, as indicated by the red circles in the figure, small misclassified regions still appear within the larger blocks, disrupting the overall consistency. MTGC outperforms the other models in both segmentation and coherence, delivering more precise region division without such misclassifications, thereby significantly improving the overall continuity and connectivity.

Figure 9 shows a quantitative evaluation of NMI and ARI, and the performance our model is further confirmed. Our method achieves the highest NMI score, indicating higher mutual information with the true partition. In terms of ARI, our method also scores highest, showing that our clustering results closely match the actual regional distribution. These results demonstrate the significant advantages of our model in capturing the functional patterns in the different areas of Manhattan.

5.4. Dynamic Downstream Task Performance

To evaluate the applicability of regional representations in dynamic tasks, we designed a bike-sharing demand forecasting task. The goal of this task is



(a) Manhattan

(b) San Francisco

Figure 9: Results of NMI and ARI.

to predict the bike usage in each region for the next hour, reflecting the short-term mobility patterns of people within the city. We utilized a real-world bike-sharing dataset from New York City [4], which includes hourly demand data for 180 regions over a 24-hour period, along with the corresponding regional representations.

The experimental pipeline adopts a sliding window approach to construct model inputs. Each sample consists of regional representations and historical demand data over several consecutive hours, which are used to predict demand in the immediately following time step. The entire prediction process is modeled at the regional granularity, and an LSTM-based architecture is employed to capture the temporal evolution trends within each region.

We evaluated the experimental results from multiple perspectives, using metrics such as MAE, RMSE, DTW distance, Pearson correlation coefficient(PPMCC), and the Top-10 high-demand region hit rate to comprehensively assess the model's performance in terms of numerical accuracy, trend consistency, and hotspot region identification.

Table 5 shows the performance of all models on the bike-sharing demand prediction task. Overall, MTGC achieves the best results across all five metrics, indicating its strong capability in learning predictive and temporally-

[4] https://citibikenyc.com

Table 5: Performance of bike-sharing demand prediction.

model	MAE ↓	RMSE ↓	DTW Distance ↓	PPMCC ↑	Top-10 Hit Rate ↑
MVURE [21]	293.65	568.26	6187.24	0.79	0.44
MGFN [9]	290.08	529.45	5739.75	0.83	0.48
HREP [24]	301.18	576.39	6261.71	0.77	0.46
MTGC	**269.60**	**442.59**	**5097.63**	**0.90**	**0.64**

aware regional representations. MVURE and HREP show relatively close performance, both capturing meaningful spatial structures through graph-based modeling. However, their limited consideration of temporal dynamics may restrict their effectiveness in highly time-sensitive tasks such as hourly demand forecasting. In contrast, MGFN achieves stronger results, particularly benefiting from its integration of multiple mobility graphs and attention-based fusion mechanisms, which allow it to encode more dynamic and multi-faceted mobility patterns. The further improvements observed with MTGC suggest that explicitly modeling region-level temporal evolution helps generate representations that are better aligned with the short-term fluctuations in urban demand.

5.5. Model Robustness Study

5.5.1. Impact of Parameter Values

To gain a deeper understanding of our model's response to different hyper-parameters, we conducted a sensitivity analysis by adjusting key parameters such as the learning rate, hidden dimension, and temperature. This analysis was performed across multiple metrics, including R^2_{Crime}, $R^2_{Check-in}$ and NMI, with the goal of evaluating the sensitivity of these parameters by recording the performance metrics for each task.

Learning rate Figure 10 demonstrates that a learning rate of 0.01 yields the best performance for both crime prediction and check-in prediction tasks, with values of R^2_{Crime} of 0.71 and $R^2_{Check-in}$ of 0.81, respectively. This indicates that a moderate learning rate effectively balances the update speed with the stability of convergence. In contrast, a learning rate of 0.1 significantly degrades performance, which is likely to be due to the large step size preventing effective convergence. When the learning rate is reduced to 0.005, the performance remains acceptable, although the slower convergence limits the overall effectiveness compared to a value of 0.01. Thus, 0.01 is the optimal choice, as it ensures efficient updates while avoiding convergence to sub-optimal solutions.

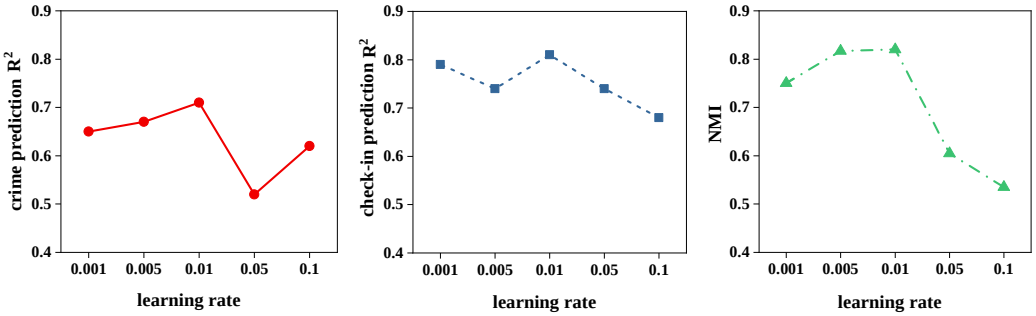


Figure 10: Impact of Learning Rate of our model.

Hidden dimension Figure 11 shows that a hidden dimension of 32 achieves the best performance for crime prediction tasks, with R^2_{Crime} reaching 0.68, while increasing the dimension to 64 or 128 does not lead to significant improvements. This suggests that a smaller hidden dimension is sufficient to capture the main patterns in the task. However, when the hidden dimension is increased to 512, the model's performance declines, with R^2_{Crime} dropping to 0.58, indicating signs of overfitting. This implies that a larger hidden dimension increases the complexity of the model, which in turn weakens its generalization ability.

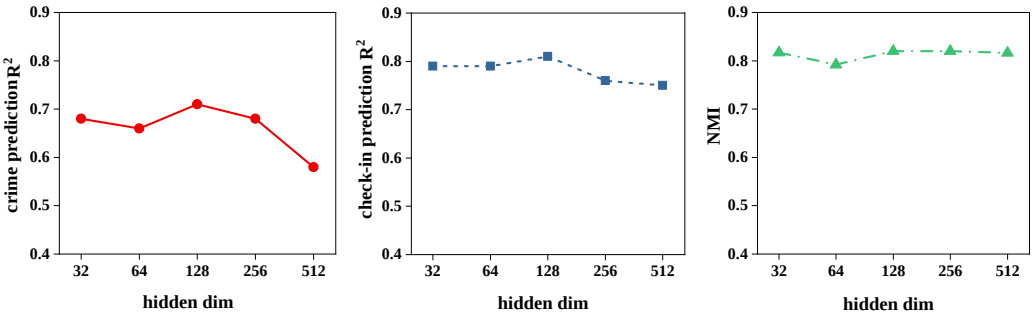


Figure 11: Impact of Hidden Dimension of our model.

Temperature Figure 12 illustrates that a temperature setting of 0.02 produces the best results across all tasks, particularly for crime and check-in predictions, with R^2 reaching 0.71 and 0.81, respectively. An appropriate temperature setting allows the model to effectively adjust the smoothness of the output probability distribution, thus improving its ability to distinguish between different categories. In contrast, lower temperatures may cause the

model to ignore complex patterns in the data, while higher temperatures lead to overly smooth distributions, reducing the model's discriminative power. Hence, a temperature of 0.02 is the optimal setting, as it strikes a balance between complexity and robustness.

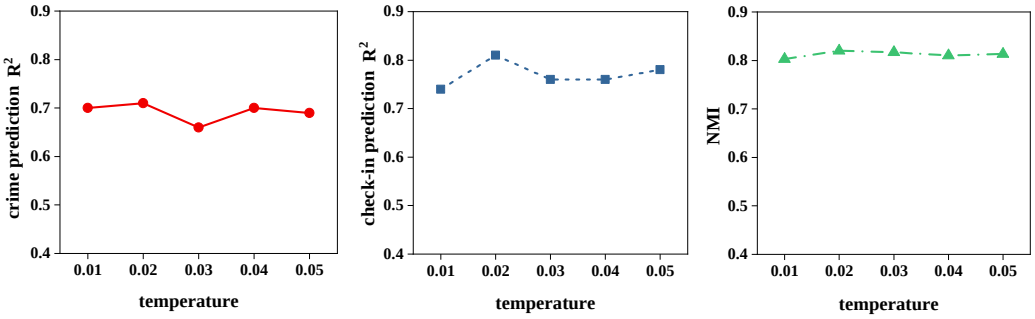


Figure 12: Impact of Temperature of our model.

This sensitivity analysis indicates that the learning rate and hidden dimension have a significant impact on model performance, while the temperature has a relatively smaller effect but still requires adjustment based on task complexity to balance generalization with feature complexity.

5.5.2. Impact of Regressors

Regression models vary in their capabilities for feature selection, nonlinear fitting, computational complexity, and robustness to noise. These differences impact the generalization ability and computational efficiency of predictive tasks, thereby affecting the stability and applicability of experimental results. To assess the sensitivity of regression models to experimental outcomes, we evaluated Lasso [34], Ridge [36], Random Forest [37], XGBoost [38], and Support Vector Regression [39].

- **Lasso Regression:** Lasso regression performs linear regression with L1 regularization, which encourages sparsity in the model by driving some coefficients to zero.
- **Ridge Regression:** Ridge regression performs linear regression with L2 regularization, which penalizes large coefficients to reduce model complexity.

- **Random Forest Regressor:** Random Forest constructs an ensemble of decision trees, where each tree is trained on a bootstrap sample and outputs are averaged to produce the final prediction.
- **XGBoost Regressor:** XGBoost is a gradient boosting algorithm that builds additive decision trees in a sequential manner, where each tree corrects the errors of the previous ones.
- **Support Vector Regression:** SVR maps input features into a high-dimensional space via kernel functions and fits a linear function within a predefined margin of tolerance to predict continuous outcomes.

Table 6: Performance of different regression models on Manhattan crime prediction. Training time refers to the duration required for 2000 iterations on an NVIDIA RTX 4090 GPU.

Regression Model	MAE ↓	RMSE ↓	R^2 ↑	Training Time ↓
Ridge Regression [36]	59.01	91.33	0.69	**1122.64s**
Random Forest [37]	67.45	91.31	0.61	9403.31s
XGBoost [38]	64.53	90.49	0.62	5780.81s
Support Vector Regression [39]	65.86	92.76	0.60	1215.94s
Lasso Regression [34]	**58.29**	**80.15**	**0.71**	1208.91s

As shown in Table 6, Lasso regression achieves the best overall performance with the lowest MAE and RMSE and the highest R^2, indicating strong predictive power. Ridge regression is the fastest, but its lack of feature selection limits its predictive accuracy. Random Forest and XGBoost incur substantially higher computational costs while delivering inferior results, possibly due to overfitting or suboptimal hyperparameter settings. Support Vector Regression performs poorly across all metrics, which may stem from its sensitivity to hyperparameters and limited ability to handle noisy, high-dimensional data. Taking both predictive accuracy and computational efficiency into account, Lasso regression emerges as the most balanced and robust choice.

5.5.3. Impact of Time Interval Partitioning at the Daily Scale

Temporal encoding plays a crucial role in capturing periodic patterns within sequential data. In this study, we incorporate time-specific embeddings to represent variations across different periods of the day. The choice

of how to partition a day into discrete time intervals can significantly affect model performance. To investigate the impact of different time interval definitions on model performance, we designed five distinct partitioning strategies, ranging from no partitioning to fine-grained peak-hour definitions. Table 7 provides a summary of these partitioning schemes.

Table 7: Different Time Partitioning Schemes.

Scheme	Morning Peak	Daytime	Evening Peak	Nighttime	Time Embedding Used
No Partitioning (NP)	-	-	-	-	No
Day & Night (DN)	-	06:00–18:00	-	18:00–06:00	Yes
Equal Partitioning (EP)	06:00–12:00	12:00–18:00	18:00–00:00	00:00–06:00	Yes
Wide Peak (WP)	06:00–10:00	10:00–18:00	18:00–22:00	22:00–06:00	Yes
MTGC Partitioning (MP)	07:00–09:00	09:00–16:00	16:00–19:00	19:00–07:00	Yes

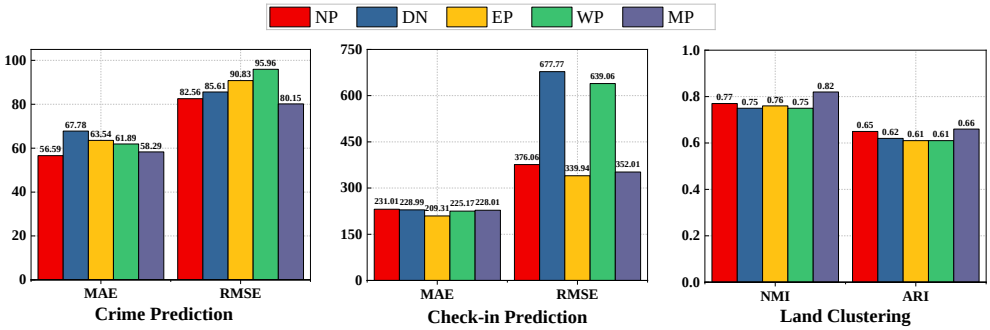


Figure 13: Performance of different partition scheme.

Figure 13 presents the experimental results of different time segmentation strategies across three tasks. Overall, MP achieves the best performance in crime prediction and region clustering, and remains highly competitive in check-in prediction, demonstrating strong generalization capability.

Specifically, although NP does not model temporal information explicitly, it still performs reasonably well in crime prediction and clustering. This suggests that improperly designed time partitioning and encoding may introduce noise rather than useful signals. DN adopts a coarse two-interval division, leading to unstable performance due to its inability to reflect actual behavioral variations. EP achieves the best result in check-in prediction, indicating that uniform segmentation can help capture regular activity patterns, though it lacks semantic alignment in other tasks. WP considers peak periods to some extent and performs moderately overall, but the broad intervals may dilute temporal distinctions. In contrast, MP leverages human

activity patterns for partitioning, enabling more precise temporal modeling and consistently strong performance across all tasks.

5.6. Ablation Study

To verify the effectiveness of each module in our proposed model, we conduct systematic ablation experiments. Through an analysis of the respective effects of each scale, we evaluate the impact of each level model and the fusion module on the overall performance.

- **MTGC-Day:** We extract representations by modeling exclusively at the day scale.
- **MTGC-Week:** We generate representations through modeling at the week scale.
- **MTGC-Month:** We derive representations by modeling solely at the month scale.
- **MTGC-AVG:** In this configuration, we replace the multi-scale contrastive learning scheme with an averaged representation obtained by modeling across the day, week, and month scales.

Table 8: Performance of ablation experiment.

Method	Crime Prediction			Check-in Prediction			Land Clustering	
	MAE↓	RMSE↓	R^2↑	MAE↓	RMSE↓	R^2↑	NMI↑	ARI↑
MTGC-Day	62.15	83.12	0.68	254.36	371.84	0.79	0.695	0.48
MTGC-Week	56.68	78.57	0.72	267.74	415.89	0.74	0.75	0.59
MTGC-Month	62.31	83.03	0.68	271.83	414.69	0.74	**0.856**	**0.80**
MTGC-Avg	**55.44**	**75.73**	**0.74**	244.10	378.38	0.79	0.684	0.51
MTGC	58.29	80.15	0.71	**228.01**	**352.04**	**0.81**	0.82	0.66

Table 8 presents the ablation outcomes for our model and its variants, leading to the following insights. (i) The day scale model demonstrates strong performance in crime prediction, while the week scale model excels in check-in prediction. These models are particularly effective at capturing temporal heterogeneity and regional variations, thereby improving their predictive capabilities for tasks involving time fluctuations. (ii) The month scale model

captures the global spatial structure of the city by integrating spatial information across regions, resulting in superior performance in land clustering tasks. This integration allows the model to identify broader patterns and relationships across regions, and to achieve high accuracy and consistency in clustering. (iii) When multi-scale contrastive learning is replaced by simple averaging, prediction tasks benefit from the stability of the blended representations. However, the results of clustering task shows a significant decline in performance, as the averaging process dilutes the spatial specificity crucial for accurate clustering.

Although models based on different time scales have particular strengths and weaknesses for specific tasks, the MTGC model effectively integrates multi-scale representations to balance these complementary advantages. The MTGC-Avg results further validate the effectiveness of the proposed multi-scale contrastive learning module, as they show that simple averaging cannot fully harness the unique contributions of each temporal scale. This demonstrates the superiority of our fusion strategy in terms of creating a model that excels across diverse tasks by leveraging the distinct advantages of each scale.

Moreover, while MTGC provides a balanced solution across tasks through multi-scale integration, individual scale models can be selectively employed in single-task scenarios to better exploit their specialized strengths. For example, day and week scale models are preferable for tasks that demand fine-grained temporal sensitivity, whereas the month scale model is more effective for capturing stable spatial structures. This flexible strategy of adapting temporal granularity to task-specific requirements enhances both the interpretability and practical applicability of the model in diverse real-world settings.

5.7. Case Study

In this case study, we aim to evaluate our model's ability to distinguish between different functional areas and socioeconomic characteristics within the city. The visual representation in Figure 14 includes a geographical map with administrative labels on the left, a 2D PCA [40] projection of region embeddings in the center, and a heatmap depicting the regional crime situation on the right. These visualizations are used to explore the correlation between the model's learned representations and real-world urban characteristics.

To investigate urban land use functions, we selected region B as the target area and chose two comparison samples. Region A, which differs functionally

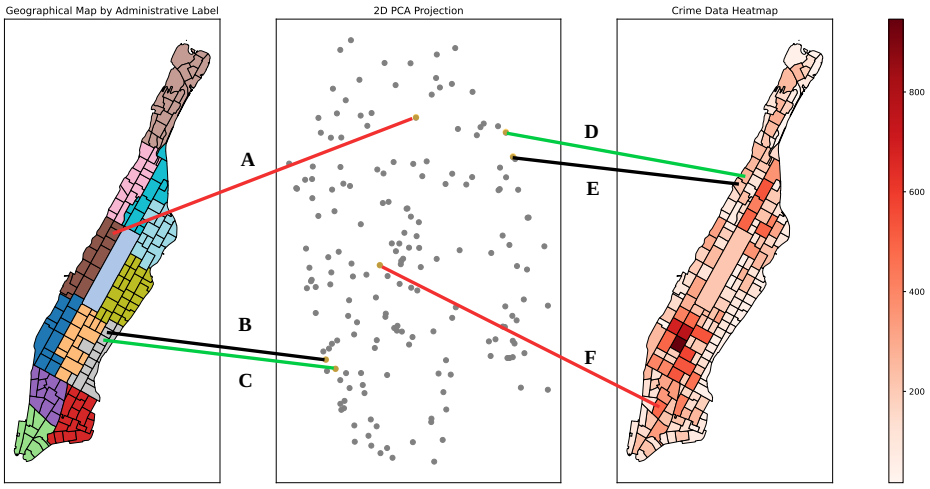


Figure 14: Cases of PCA and Geographic Distance and Crime Situation.

from region B, is connected by a red line between its geographical location and its PCA projection. Region C, which is functionally similar to region B, is connected by a green line. The black line links region B's geographical location to its position in the PCA projection. The figure shows that regions A and B are far apart in the embedding space, highlighting their functional differences, while regions B and C are much closer, indicating similar functional characteristics.

Next, we examined the relationship between crime levels and the model's representations. We selected regions D, E, and F for this comparison. Regions D and E, which have similar crime rates, are connected by green and black lines to their respective positions in both the crime heatmap and PCA space. In contrast, region F, with a higher crime rate, is connected by the red line and is much farther from region E in the PCA projection. This demonstrates the model's ability to reflect regional crime differences in the embedding space.

From this analysis, it is clear that our model effectively distinguishes between regions based on functional and socioeconomic characteristics while maintaining consistency for regions with similar traits.

6. Conclusion

In this paper, we present MTGC, an unsupervised learning framework designed to capture comprehensive urban region representations by integrating multi-scale spatio-temporal information. The framework models daily and weekly mobility dynamics to capture local temporal heterogeneity, while a heterogeneous spatial graph at the month scale is employed to learn global spatial characteristics. Through a multi-view fusion mechanism, MTGC generates robust region embeddings, which demonstrate strong utility in functional zone identification and other downstream tasks.

In future work, we plan to further improve the generality and expressiveness of the framework. First, we will explore the incorporation of additional data sources such as POI and other semantic information to enrich the representation space and enable fairer comparisons with multi-view methods like AutoST [25] and GraphST [10]. Second, we aim to develop more scientifically effective techniques for feature extraction at different temporal scales, better capturing complex variations and interactions. Third, we intend to investigate strategies that enhance multi-scale collaboration, thereby producing more cohesive and fine-grained characterizations of urban dynamics.

Acknowledgments

This research is supported by the National Natural Science Foundation of China (Grant No.62272087); Science and Technology Planning Project of Sichuan Province (Grant No.2024YFFK0116).

References

[1] U. N. D. of Economic, S. Affairs, World Urbanization Prospects, United Nations, 2018.

[2] X. Zou, Y. Yan, X. Hao, Y. Hu, H. Wen, E. Liu, J. Zhang, Y. Li, T. Li, Y. Zheng, et al., Deep learning for cross-domain data fusion in urban computing: Taxonomy, advances, and outlook, Information Fusion 113 (2025) 102606.

[3] G. Jin, H. Sha, Z. Xi, J. Huang, Urban hotspot forecasting via automated spatio-temporal information fusion, Applied Soft Computing 136 (2023) 110087.

[4] J. Cao, X. Wang, G. Chen, W. Tu, X. Shen, T. Zhao, J. Chen, Q. Li, Disentangling the hourly dynamics of mixed urban function: A multimodal fusion perspective using dynamic graphs, Information Fusion (2024) 102832.

[5] Y. Fu, P. Wang, J. Du, L. Wu, X. Li, Efficient region embedding with multi-view spatial networks: A perspective of locality-constrained spatial autocorrelations, in: Proc. AAAI, Vol. 33, 2019, pp. 906–913.

[6] W. Chan, Q. Ren, Region-wise attentive multi-view representation learning for urban region embedding, in: Proc. CIKM, 2023, pp. 3763–3767.

[7] Z. Li, W. Huang, K. Zhao, M. Yang, Y. Gong, M. Chen, Urban region embedding via multi-view contrastive prediction, in: Proc. AAAI, Vol. 38, 2024, pp. 8724–8732.

[8] H. Wang, Z. Li, Region representation learning via mobility flow, in: Proc. CIKM, 2017, pp. 237–246.

[9] S. Wu, X. Yan, X. Fan, S. Pan, S. Zhu, C. Zheng, M. Cheng, C. Wang, Multi-graph fusion networks for urban region embedding, in: Proc. IJCAI, 2022, pp. 2312–2318.

[10] Q. Zhang, C. Huang, L. Xia, Z. Wang, S. M. Yiu, R. Han, Spatial-temporal graph learning with adversarial contrastive adaptation, in: Proc. ICML, PMLR, 2023, pp. 41151–41163.

[11] G. Jin, Y. Liang, Y. Fang, Z. Shao, J. Huang, J. Zhang, Y. Zheng, Spatio-temporal graph neural networks for predictive learning in urban computing: A survey, IEEE Transactions on Knowledge and Data Engineering 36 (10) (2024) 5388–5408.

[12] J. Ou, J. Sun, Y. Zhu, H. Jin, Y. Liu, F. Zhang, J. Huang, X. Wang, Stp-trellisnets+: Spatial-temporal parallel trellisnets for multi-step metro station passenger flow prediction, IEEE Transactions on Knowledge and Data Engineering 35 (2023) 7526–7540.

[13] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: Proc. ICLR, 2017, pp. 1–14.

[14] Y. Kang, Y. Peng, D. Zheng, H. Zhang, X. Yang, Multi-view framework for multi-label bioactive peptide classification based on multi-modal representation learning, Applied Soft Computing 175 (2025) 113007.

[15] G. Teng, T. Mao, C. Shen, X. Tian, X. Liu, Y. Chen, J. Ye, Urrl-imvc: Unified and robust representation learning for incomplete multi-view clustering, in: Proc. SIGKDD, 2024, p. 2888–2899.

[16] Z. Yao, Y. Fu, B. Liu, W. Hu, H. Xiong, Representing urban functions through zone embedding with human mobility patterns, in: Proc. IJCAI, 2018, pp. 3919–3925.

[17] Y. Zhang, Y. Fu, P. Wang, X. Li, Y. Zheng, Unifying inter-region autocorrelation and intra-region structures for spatial embedding via collective adversarial learning, in: Proc. SIGKDD, 2019, pp. 1700–1708.

[18] Z. Wang, H. Li, R. Rajagopal, Urban2vec: Incorporating street view imagery and pois for multi-modal urban neighborhood embedding, in: Proc. AAAI, Vol. 34, 2020, pp. 1013–1020.

[19] Y. Yuan, Z. Li, B. Zhao, A survey of multimodal learning: Methods, applications, and future, ACM Computing Surveys (2025).

[20] Y. Luo, F. lai Chung, K. Chen, Urban region profiling via multi-graph representation learning, in: Proc. CIKM, 2022, pp. 4294–4298.

[21] M. Zhang, T. Li, Y. Li, P. Hui, Multi-view joint graph representation learning for urban region embedding, in: Proc. IJCAI, 2021, pp. 4431–4437.

[22] L. Zhang, C. Long, G. Cong, Region embedding with intra and inter-view contrastive learning, IEEE Transactions on Knowledge and Data Engineering 35 (9) (2022) 9031–9036.

[23] F. Sun, J. Qi, Y. Chang, X. Fan, S. Karunasekera, E. Tanin, Urban region representation learning with attentive fusion, in: Proc. ICDE, IEEE, 2024, pp. 4409–4421.

[24] S. Zhou, D. He, L. Chen, S. Shang, P. Han, Heterogeneous region embedding with prompt learning, in: Proc. AAAI, Vol. 37, 2023, pp. 4981–4989.

[25] Q. Zhang, C. Huang, L. Xia, Z. Wang, Z. Li, S. Yiu, Automated spatio-temporal graph contrastive learning, in: Proc. WWW, 2023, pp. 295–305.

[26] P. Brantingham, P. Brantingham, Criminality of place: Crime generators and crime attractors, European journal on criminal policy and research 3 (1995) 5–26.

[27] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, Advances in neural information processing systems 30 (2017).

[28] R. Ni, Z. Lin, S. Wang, G. Fanti, Mixture-of-linear-experts for long-term time series forecasting, in: International Conference on Artificial Intelligence and Statistics, PMLR, 2024, pp. 4672–4680.

[29] A. Zeevi, R. Meir, R. Adler, Time series prediction using mixtures of experts, Advances in neural information processing systems 9 (1996).

[30] G. Jin, C. Liu, Z. Xi, H. Sha, Y. Liu, J. Huang, Adaptive dual-view wavenet for urban spatial–temporal event prediction, Information Sciences 588 (2022) 315–330.

[31] T. N. Kipf, M. Welling, Variational graph auto-encoders, stat 1050 (2016) 21.

[32] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, Q. Mei, Line: Large-scale information network embedding, in: Proc. WWW, 2015, pp. 1067–1077.

[33] A. Grover, J. Leskovec, node2vec: Scalable feature learning for networks, in: Proc. SIGKDD, 2016, pp. 855–864.

[34] R. Tibshirani, Regression shrinkage and selection via the lasso, Journal of the Royal Statistical Society Series B: Statistical Methodology 58 (1) (1996) 267–288.

[35] S. Lloyd, Least squares quantization in pcm, IEEE Transactions on Information Theory 28 (1982) 129–137.

[36] A. E. Hoerl, R. W. Kennard, Ridge regression: Biased estimation for nonorthogonal problems, Technometrics 12 (1) (1970) 55–67.

[37] L. Breiman, Random forests, Machine learning 45 (2001) 5–32.

[38] T. Chen, C. Guestrin, Xgboost: A scalable tree boosting system, in: Proc. SIGKDD, 2016, pp. 785–794.

[39] H. Drucker, C. J. Burges, L. Kaufman, A. Smola, V. Vapnik, Support vector regression machines, Advances in neural information processing systems 9 (1996).

[40] I. T. Jolliffe, J. Cadima, Principal component analysis: a review and recent developments, Philosophical transactions of the royal society A: Mathematical, Physical and Engineering Sciences 374 (2065) (2016) 20150202.